

Subscripts

Introduction

- Classes, structures and enumeration can define subscripts which are **shortcuts for accessing the member elements of a collection, list or sequence**.
- You use subscripts to **set and retrieve values by index** without needing separate methods for setting and retrieval.
- For example, you access elements in an Array instance as **someArray[index]** and elements in a Dictionary instance as **someDictionary[key]**.
- You can **define multiple subscripts for a single type** and the appropriate **subscript overload** to use selected based on the type of index value you pass to the subscript.
- Subscripts aren't limited to a single dimension and you can define subscripts with multiple input parameters to suite your custom type's needs.

Subscripts Syntax

- Subscripts enable you to **query instances of a type** by writing one or more values in square brackets after the instance name.
- Their syntax is similar to both instance method syntax and computed property syntax.
- You write subscript definitions with the **subscript keyword** and specify one or more input parameters and return type in the same way as instance methods.
- Unlike instance methods, subscripts can be **read-write** and **read-only**.

Subscript Syntax

```
subscript(index: Int) -> Int {  
    get{  
        //Return an appropriate subscript value here.  
    }  
    set(newValue) {  
        //Perform a suitable setting action here.  
    }  
}
```

- The type of `newValue` is the same as the return value of the subscript.

Example

- An example of a read-only subscript implementation, which defines a TimesTable structure to represent an n-times-table of integers

```
1  struct TimesTable {  
2      let multiplier: Int  
3      subscript(index: Int) -> Int {  
4          return multiplier * index  
5      }  
6  }  
7  let threeTimesTable = TimesTable(multiplier:  
    3)  
8  print("six times three is \  
    (threeTimesTable[6])")  
9  // Prints "six times three is 18"
```

Example

```
class MyList {  
    var items: [String]  
  
    init(items: [String]) {  
        self.items = items  
    }  
  
    subscript(index: Int) -> String {  
        get {  
            return items[index]  
        }  
        set {  
            items[index] = newValue  
        }  
    }  
}
```

Subscript Options

- Subscripts can take **any number of input parameters**, and these **input parameters can be of any type**. Subscripts can also **return a value of any type**.
- Like functions, subscripts can take a varying number of parameters and provide default values for their parameters.
- A class or structure can provide as many subscript implementations as it needs, and the appropriate subscript to be used will be inferred based on the types of the value or values that are contained within the subscript brackets at the point that the subscript is used. This definition of multiple subscripts is known as *subscript overloading*.
- It's most common for a subscript to take a single parameter, you can also define a subscript with multiple parameters if it's appropriate for your type.